Event ends in 2 days 2 hours 55 minutes.

Security infrastructure forms the foundation of your chatbot application's defense strategy. This section creates comprehensive security specifications covering network security, identity management, data protection, and compliance controls.

Exercise 2.1: Network Security and Access Controls

Design comprehensive network security architecture:

~~~
Create detailed network security specifications for our chatbot infrastructure:

**CloudFront Security Configuration**:
Use the AWS documentation MCP server to research current CloudFront security best practices, then specify:

**Web Application Firewall (WAF) Integration**:
- **WAF Web ACL**: `chatbot-waf-${Environment}`
- **Rule Groups**:
  - AWS Managed Rules - Core Rule Set
  - AWS Managed Rules - Known Bad Inputs
  - AWS Managed Rules - SQL Injection Prevention
  - AWS Managed Rules - Cross-Site Scripting (XSS) Prevention
- **Custom Rules**:
  - Rate limiting: 1000 requests per 5 minutes per IP
  - Geographic restrictions: Block high-risk countries
  - IP reputation: Block known malicious IPs
  - Bot control: Challenge suspicious automated traffic

**Security Headers Configuration**:
```json
{
  "Strict-Transport-Security": "max-age=31536000; includeSubdomains; preload",
  "Content-Security-Policy": "default-src 'self'; connect-src 'self' https://*.execute-api.${AWS::Region}.amazonaws.com https://*.amazonaws.com; script-src 'self'; style-src 'se
  "X-Content-Type-Options": "nosniff",
  "X-Frame-Options": "DENY",
  "X-XSS-Protection": "1; mode=block",
  "Referrer-Policy": "strict-origin-when-cross-origin",
  "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()"
}
```

**SSL/TLS Configuration**:
- **Minimum TLS Version**: 1.2
- **Cipher Suites**: Modern, secure cipher suites only
- **Certificate**: AWS Certificate Manager with auto-renewal
- **HSTS**: Enabled with preload directive
- **Certificate Transparency**: Enabled for monitoring

**API Gateway Security**:
- **Resource Policies**: Restrict access to CloudFront and authorized IPs
- **Request Validation**: Strict input validation for all endpoints
- **Throttling**: Per-client and per-method rate limiting
- **API Keys**: Required for all non-authentication endpoints
- **Usage Plans**: Tiered access with quotas and throttling

**Network Access Control**:
- **VPC Configuration** (if needed for enhanced security):
  - Private subnets for Lambda functions
  - NAT Gateway for outbound internet access
  - VPC Endpoints for AWS services
  - Security Groups with minimal required access
- **Network ACLs**: Additional layer of network security
- **Flow Logs**: VPC flow logs for network monitoring and analysis

Research current AWS network security best practices and WAF configurations using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive network security architecture with multiple layers of protection
- WAF configuration protecting against common web attacks
- Proper SSL/TLS configuration with modern security standards
- API Gateway security controls with rate limiting and validation
- Network isolation and access controls for enhanced security

Exercise 2.2: Identity and Access Management Security

Create comprehensive IAM security framework:

~~~
Design detailed IAM security specifications following AWS security best practices:

**IAM Security Policies and Controls**:

**Permission Boundaries**:
```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": ["${AWS::Region}"]
        },
        "StringLike": {
          "aws:userid": ["AIDACKCEVSQ6C2EXAMPLE:*"]
        }
      }
    },
    {
      "Effect": "Deny",
      "Action": [
        "iam:CreateUser",
        "iam:CreateRole",
        "iam:AttachUserPolicy",
        "iam:AttachRolePolicy",
        "iam:PutUserPolicy",
        "iam:PutRolePolicy"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:userid": ["AIDACKCEVSQ6C2EXAMPLE:admin-user"]
        }
      }
    }
  ]
}
```

**Service Control Policies (SCPs)**:
- **Deny Root User Actions**: Prevent root user access except for billing
- **Enforce MFA**: Require MFA for all privileged operations
- **Region Restrictions**: Limit operations to approved regions
- **Service Restrictions**: Prevent use of unapproved AWS services
- **Resource Tagging**: Enforce mandatory resource tagging

**Cross-Account Access Patterns**:
- **Development Account**: Limited read access to production resources
- **Audit Account**: Read-only access for compliance and monitoring
- **Security Account**: Centralized security tooling and monitoring
- **Backup Account**: Cross-region backup and disaster recovery

**IAM Access Analyzer Configuration**:
- **Analyzer Name**: `chatbot-access-analyzer-${Environment}`
- **Scope**: Account-wide analysis
- **Findings**: Monitor for unintended external access
- **Archive Rules**: Automatically archive expected findings
- **Integration**: CloudWatch Events for real-time notifications

**AWS Organizations Security Configuration**:
- **Service Control Policies**: Centralized security controls
- **Config Rules**: Compliance monitoring across accounts
- **CloudTrail**: Centralized audit logging
- **GuardDuty**: Threat detection across organization
- **Security Hub**: Centralized security findings management

**Identity Federation** (if applicable):
- **SAML 2.0 Integration**: Corporate identity provider integration
- **OIDC Integration**: Modern authentication protocols
- **Attribute-Based Access Control**: Fine-grained permissions based on user attributes
- **Just-in-Time Access**: Temporary elevated permissions
- **Session Duration**: Configurable session timeouts

Research current IAM security best practices, including AWS Organizations security features and identity federation patterns using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive IAM security framework with permission boundaries and SCPs
- Cross-account access patterns for multi-environment security
- IAM Access Analyzer configuration for continuous monitoring
- AWS Organizations security controls for centralized management
- Identity federation patterns for enterprise integration

Exercise 2.3: Data Protection and Encryption

Design comprehensive data protection and encryption strategies:

~~~
Create detailed data protection specifications covering all data states and flows:

**Encryption at Rest Configuration**:

**DynamoDB Encryption**:
- **Encryption Type**: Customer-managed KMS keys
- **Key Rotation**: Automatic annual rotation
- **Key Policy**:
```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Enable IAM User Permissions",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::${AWS::AccountId}:root"
      },
      "Action": "kms:*",
      "Resource": "*"
    },
    {
      "Sid": "Allow DynamoDB Service",
      "Effect": "Allow",
      "Principal": {
        "Service": "dynamodb.amazonaws.com"
      },
      "Action": [
        "kms:Decrypt",
        "kms:GenerateDataKey"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "kms:ViaService": "dynamodb.${AWS::Region}.amazonaws.com"
        }
      }
    }
  ]
}
```

**S3 Encryption**:
- **Default Encryption**: AES-256 (SSE-S3) for frontend assets
- **Sensitive Data**: Customer-managed KMS keys for logs and backups
- **Bucket Key**: Enabled for cost optimization
- **Encryption in Transit**: HTTPS only with SSL/TLS 1.2+

**Secrets Manager Encryption**:
- **Encryption**: Customer-managed KMS keys
- **Automatic Rotation**: 90-day rotation for JWT secrets
- **Cross-Region Replication**: For disaster recovery
- **Access Logging**: All secret access logged to CloudTrail

**Lambda Environment Variables**:
- **Encryption**: Customer-managed KMS keys
- **Sensitive Data**: Store in Secrets Manager, not environment variables
- **Access Patterns**: Retrieve secrets at runtime, not startup

**Encryption in Transit Configuration**:
- **API Gateway**: TLS 1.2+ with modern cipher suites
- **CloudFront**: TLS 1.2+ with HSTS enforcement
- **Internal Communications**: All AWS service communications encrypted
- **Database Connections**: Encrypted connections to DynamoDB
- **Certificate Management**: AWS Certificate Manager with auto-renewal

**Data Classification and Handling**:
- **Public Data**: Marketing content, public documentation
- **Internal Data**: Application logs, metrics, non-sensitive configuration
- **Confidential Data**: User profiles, conversation metadata
- **Restricted Data**: Authentication credentials, personal information
- **Data Loss Prevention**: Automated scanning for sensitive data patterns

**Key Management Strategy**:
- **Key Hierarchy**: Master keys, data keys, and envelope encryption
- **Key Rotation**: Automatic rotation with configurable schedules
- **Key Backup**: Cross-region key replication for disaster recovery
- **Key Access**: Least privilege access with detailed audit logging
- **Key Monitoring**: CloudWatch metrics and alarms for key usage

Research current AWS encryption best practices and key management strategies using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive encryption strategy for all data states and flows
- Customer-managed KMS keys with proper policies and rotation
- Data classification framework with appropriate protection levels
- Key management strategy with backup and monitoring
- Compliance with data protection regulations and standards

Exercise 2.4: Compliance and Governance Framework

Create comprehensive compliance and governance specifications:

~~~
Design detailed compliance and governance framework for our chatbot application:

**GDPR Compliance Implementation**:

**Data Subject Rights**:
- **Right to Access**: API endpoints for users to retrieve their data
- **Right to Rectification**: User profile update functionality
- **Right to Erasure**: Complete data deletion with verification
- **Right to Portability**: Data export in machine-readable format
- **Right to Restrict Processing**: Temporary data processing suspension

**Privacy by Design Implementation**:
```json
{
  "DataMinimization": {
    "UserData": "Collect only necessary information for chatbot functionality",
    "ConversationData": "Store only essential conversation context",
    "LoggingData": "Log only non-personal identifiable information"
  },
  "PurposeLimitation": {
    "DataUsage": "Use data only for stated chatbot purposes",
    "ThirdPartySharing": "No data sharing without explicit consent",
    "Analytics": "Anonymized data only for service improvement"
  },
  "StorageLimitation": {
    "RetentionPeriod": "Delete user data after 2 years of inactivity",
    "AutomaticDeletion": "Automated cleanup processes with audit trails",
    "BackupRetention": "Encrypted backups with same retention policies"
  }
}
```

**SOC 2 Type II Controls**:

**Security Controls**:
- **Access Controls**: Multi-factor authentication for all administrative access
- **Network Security**: Firewall rules and intrusion detection systems
- **Vulnerability Management**: Regular security assessments and patch management
- **Incident Response**: Documented procedures with defined response times

**Availability Controls**:
- **System Monitoring**: 24/7 monitoring with automated alerting
- **Backup Procedures**: Regular backups with tested restore procedures
- **Disaster Recovery**: Documented recovery procedures with RTO/RPO targets
- **Change Management**: Controlled deployment processes with rollback capabilities

**Processing Integrity Controls**:
- **Data Validation**: Input validation and sanitization for all user inputs
- **Error Handling**: Comprehensive error handling with logging
- **Transaction Processing**: Atomic operations with consistency checks
- **Audit Logging**: Complete audit trail for all system activities

**Confidentiality Controls**:
- **Data Encryption**: Encryption at rest and in transit for all sensitive data
- **Access Logging**: Detailed logging of all data access activities
- **Data Classification**: Clear classification and handling procedures
- **Secure Development**: Security code review and testing procedures

**Privacy Controls**:
- **Consent Management**: Clear consent mechanisms with audit trails
- **Data Minimization**: Collection and processing of minimal necessary data
- **Purpose Limitation**: Data use limited to stated purposes
- **Individual Rights**: Mechanisms for exercising privacy rights

**Audit and Monitoring Framework**:
- **CloudTrail Configuration**: All API calls logged with integrity protection
- **Config Rules**: Continuous compliance monitoring with automated remediation
- **Security Hub**: Centralized security findings with prioritization
- **GuardDuty**: Threat detection with automated response
- **Inspector**: Vulnerability assessments for applications and infrastructure

**Compliance Reporting**:
- **Automated Reports**: Daily, weekly, and monthly compliance status reports
- **Audit Preparation**: Automated evidence collection for audits
- **Remediation Tracking**: Automated tracking of compliance issues and resolution
- **Metrics and KPIs**: Compliance metrics with trend analysis

Research current compliance frameworks and AWS compliance services using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive GDPR compliance implementation with privacy by design
- SOC 2 Type II controls framework with detailed procedures
- Automated compliance monitoring and reporting
- Audit and evidence collection processes
- Continuous compliance improvement framework

Exercise 2.5: Security Monitoring and Incident Response

Design comprehensive security monitoring and incident response capabilities:

~~~
Create detailed security monitoring and incident response specifications:

**Security Information and Event Management (SIEM)**:

**CloudWatch Security Monitoring**:
- **Custom Metrics**:
  - Failed authentication attempts per minute
  - API error rates by endpoint
  - Unusual traffic patterns and spikes
  - Bedrock API usage and costs
  - Database connection failures
- **Alarms and Thresholds**:
  - Critical: Failed logins > 10 per minute
  - Warning: API error rate > 5%
  - Info: Unusual traffic increase > 200%
- **Automated Responses**:
  - Temporary IP blocking for brute force attempts
  - API throttling for abuse patterns
  - Notification escalation for critical events

**AWS Security Services Integration**:
- **GuardDuty Configuration**:
  - Threat intelligence feeds enabled
  - DNS logging for malware detection
  - VPC Flow Logs analysis (if VPC is used)
  - S3 protection for data exfiltration detection
- **Security Hub Integration**:
  - Centralized security findings aggregation
  - Custom insights for chatbot-specific threats
  - Automated finding prioritization
  - Integration with ticketing systems
- **Config Rules**:
  - S3 bucket public access monitoring
  - IAM policy compliance checking
  - Encryption configuration validation
  - Security group rule monitoring

**Incident Response Procedures**:

**Incident Classification**:
- **P1 - Critical**: Data breach, service unavailable, security compromise
- **P2 - High**: Performance degradation, partial service outage
- **P3 - Medium**: Minor security issues, configuration drift
- **P4 - Low**: Informational alerts, maintenance notifications

**Response Procedures**:
```json
{
  "P1_Critical": {
    "ResponseTime": "15 minutes",
    "EscalationPath": ["Security Team", "Engineering Manager", "CTO"],
    "Actions": [
      "Isolate affected systems",
      "Preserve evidence for forensics",
      "Notify stakeholders and customers",
      "Implement containment measures",
      "Begin recovery procedures"
    ]
  },
  "P2_High": {
    "ResponseTime": "1 hour",
    "EscalationPath": ["On-call Engineer", "Team Lead"],
    "Actions": [
      "Assess impact and scope",
      "Implement temporary workarounds",
      "Begin root cause analysis",
      "Update stakeholders"
    ]
  }
}
```

**Forensics and Investigation**:
- **Evidence Preservation**: Automated snapshots and log preservation
- **Chain of Custody**: Documented evidence handling procedures
- **Analysis Tools**: CloudTrail Insights, VPC Flow Logs analysis
- **External Resources**: Third-party forensics capabilities if needed

**Communication and Notification**:
- **Internal Notifications**: Slack, email, SMS for different severity levels
- **External Notifications**: Customer communication templates
- **Regulatory Notifications**: GDPR breach notification procedures
- **Media Relations**: Public communication procedures for major incidents

**Recovery and Lessons Learned**:
- **Recovery Procedures**: Step-by-step recovery with validation checkpoints
- **Post-Incident Review**: Mandatory review within 48 hours
- **Root Cause Analysis**: Detailed analysis with corrective actions
- **Process Improvement**: Updates to procedures based on lessons learned

**Security Testing and Validation**:
- **Penetration Testing**: Quarterly external penetration tests
- **Vulnerability Scanning**: Weekly automated vulnerability scans
- **Red Team Exercises**: Annual red team exercises
- **Tabletop Exercises**: Quarterly incident response tabletop exercises

Research current security monitoring best practices and incident response frameworks using the AWS documentation MCP server.
~~~

Expected Outcome

- Comprehensive security monitoring with automated detection and response
- Detailed incident response procedures with clear escalation paths
- Forensics and investigation capabilities for security incidents
- Communication and notification procedures for all stakeholders
- Continuous improvement through testing and lessons learned

Key Deliverables

After completing the Security Infrastructure exercises, you should have:

1. **Network Security Architecture**: Comprehensive protection with WAF, security headers, and access controls
2. **IAM Security Framework**: Least privilege access with permission boundaries and monitoring
3. **Data Protection Strategy**: Encryption at rest and in transit with proper key management
4. **Compliance Framework**: GDPR and SOC 2 compliance with automated monitoring
5. **Security Monitoring and Incident Response**: Comprehensive detection, response, and recovery capabilities

Integration with Advanced Features

Your security infrastructure leverages:

- **AWS Documentation MCP Server**: For current security best practices and configurations
- **Agent Steering**: For applying security standards and compliance requirements
- **Agent Hooks**: For validating security configurations and updating documentation

Next Steps

With comprehensive security infrastructure specifications complete, you're ready to design the deployment pipeline that will safely and reliably deploy your chatbot application. The next section will focus on CI/CD automation, environment management, and deployment strategies that maintain the security posture you've established.

Previous Next